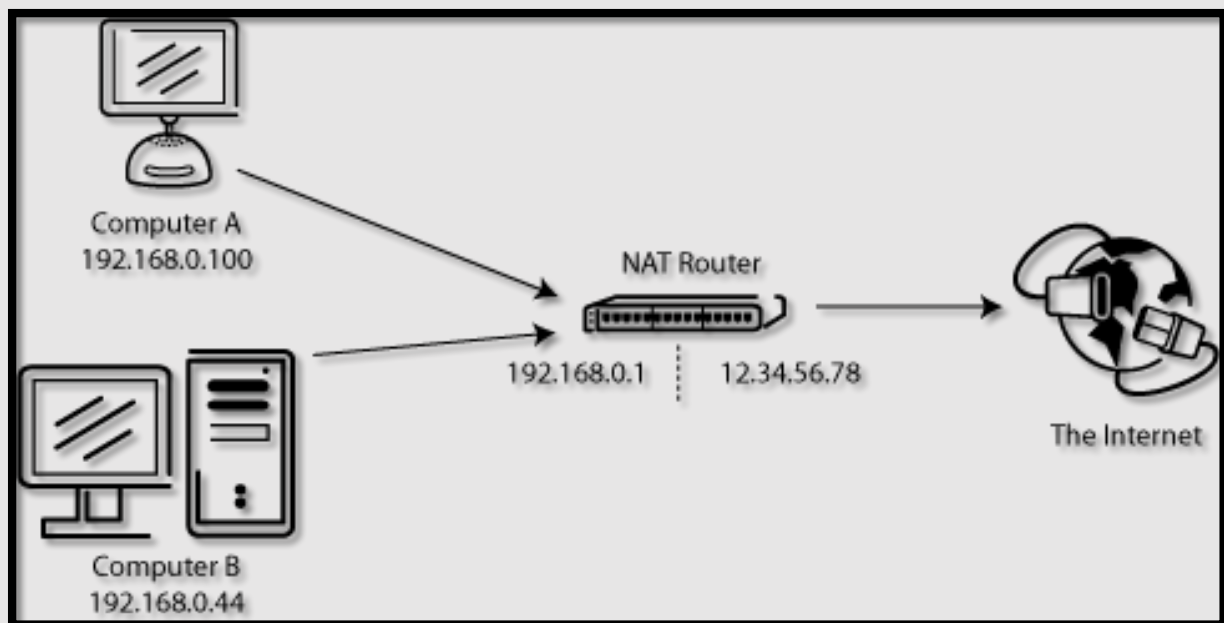


תיק פרוייקט – 5 יחידות לימוד

מימוש מערכת NAT



שם המגיש: יעקב חזון.

תעודת זהות: [REDACTED]

תאריך הגשה: 4/5/2017.

בית ספר: תיכון "אמירים", מקיף ה'.

כיתה: יב'-2.

שמות המנחים: ביצוע עצמאי.

תוכן עניינים

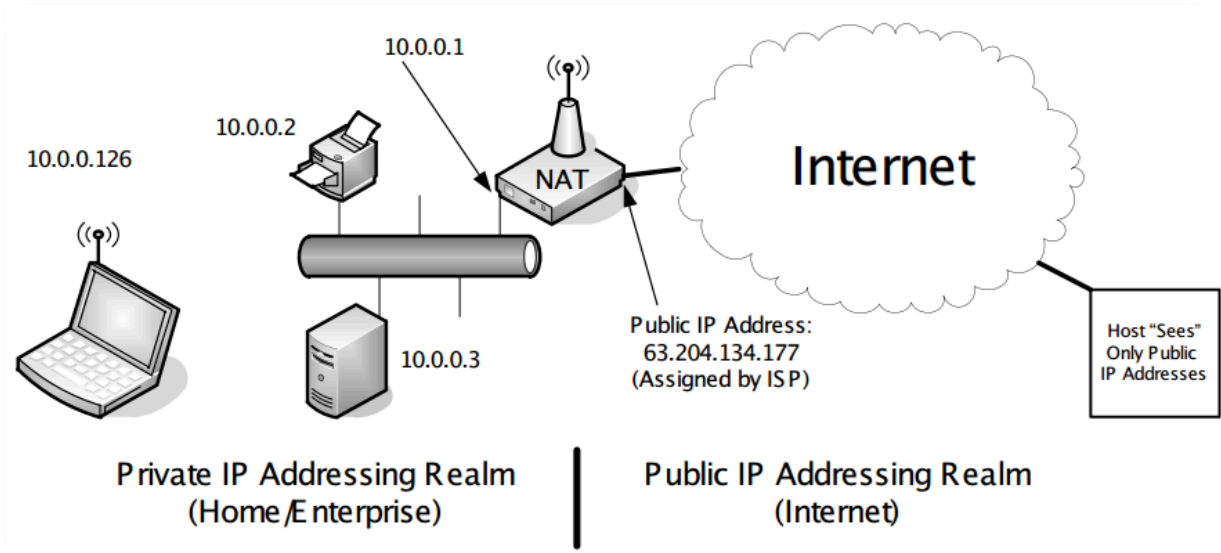
1.....	תוכן עניינים
2.....	רקע
5.....	תיאור המוצר המוגמר
7.....	תיאור הפיתוח
9.....	תיאור הממשק למשתמש
14.....	סביבת העבודה
16.....	תיאור המודולים
18.....	תיעוד הקוד ואלגוריתמים
21.....	תדפיס הקוד
47.....	ביבליוגרפיה

כאשר התחלתי לכתוב את הפרויקט היה חשוב לי לממש דבר הפותר צורך ממשי ושממנו אוכל ללמוד רבות על רשתות מחשבים וכתובה בשפת פייתון בגרסה של 2.7. בחשיבתי על נושא הפרויקט נזכרתי כי למדתי אודות כתובות IP בגרסה 4 המורכבים מארבע בתים ולפיכך ישנן 4,294,967,296 אפשרויות לכתובות שונות. הרי שישנם הרבה יותר ממספר זה של מחשבים אשר מחוברים לרשת האינטרנט ולכן היה צורך למצוא פתרון היאפשר גישת מחשבים לאינטרנט למרות ההגבלה המדוברת (הגבלה שתוקנה בגרסה השישית של פרוטוקול ה-IP, אך הוא עדיין לא בשימוש נפוץ).

לאחר חקירה מעמיקה אודות הבעיה המתוארת, נתקלתי בפתרונה הרווח ביותר והוא שימוש במערכת NAT. NAT הוא קיצור ל- Network Address Translation, כלומר, תרגום כתובות רשת. יש שימוש במערכת שכזו במרבית הנתבים הביתיים והיא נחשבת ליעילה ביותר. טכניקה זו בעצם ממירה את הכתובות של מנות מידע המגיעות לנתב או לחומת אש מסוימת. בכך, באמצעות "רגל" (ממשק אינטרנט) שיש ברשת אחת ו"רגל" אחרת שיש ברשת אחרת (או ברשת האינטרנט) לנתב ניתן לתרגם כתובות רבות של מחשבים הנמצאות ברשת אחת לכתובת ציבורית אחת שהנתב מקבל מספק שירות האינטרנט.

כאשר מגיעה מנת מידע אל נתב הרשת העושה שימוש במערכת NAT, לפי פרוטוקול התעבורה של מנת המידע, כאשר הנפוצים בהם הינם ICMP, TCP ו-UDP, אותם מימשתי בפרויקט שלי, הוא ממיר את כתובת המוצא של מנת המידע בכתובת הציבורית של עצמו (ולעיתים גם את פורט המקור כשאפשר) ושומר את המידע אודות מנת המידע והמרתה במבנה נתונים הנקרא NAT Table כדי שידע לשייך ולהחזיר את מנת התגובה בחזרה למחשב ברשת הפרטית. ניתן לראות כי בעצם באמצעות שיטה זו מספר רב של מחשבים יכולים בפועל לבצע שימוש בכתובת ציבורית אחת ויחידה ואליה מחשב היעד ברשת הציבורית יחזיר את תגובתו.

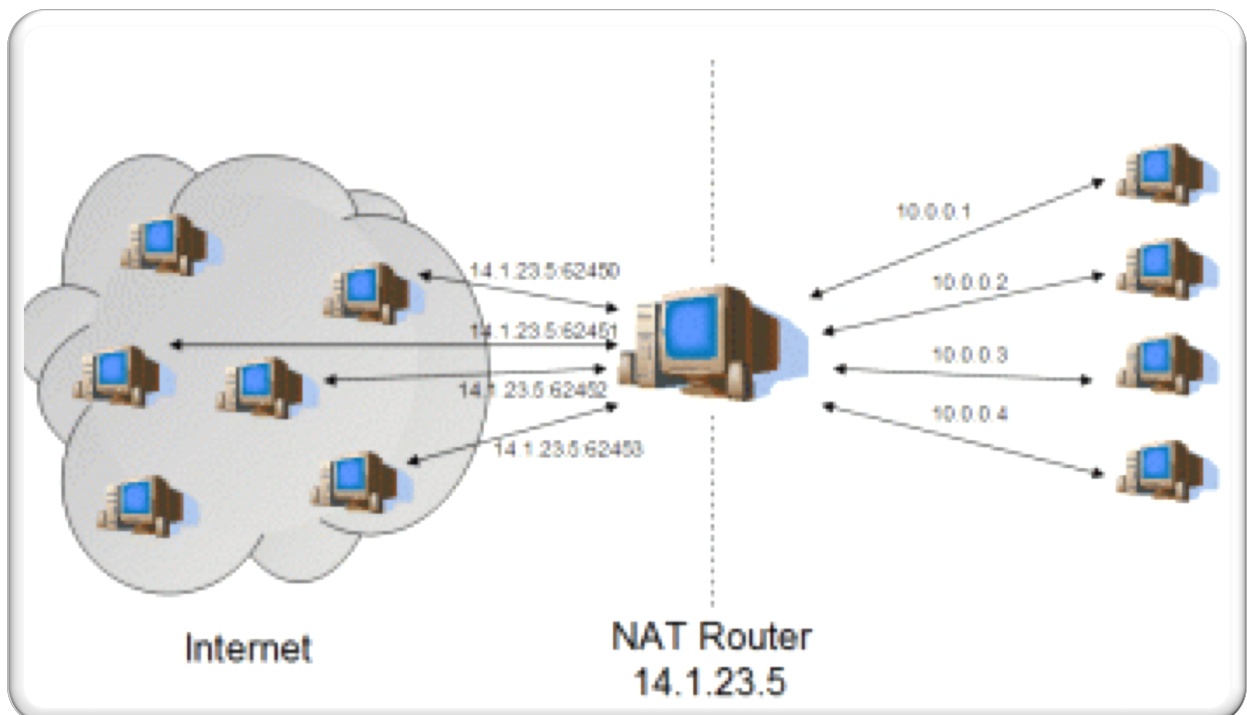
כמו כן, על המערכת העושה שימוש ב-NAT להעניק למחשבים ברשת הציבורית כתובות IP פרטיות לצורך התקשורת שלה עימם ושיוך פקטות אליהם. מערכת שכזו תוכל במקרה של חוסר שימוש להעניק את אותה הכתובת למחשב אחר ברשת הפרטית. יתרה מכך, יש ל-NAT יתרון משמעותי מבחינת בטיחות בכך שניתן להסתיר את מבנה הרשת הפנימית מפני אנשים מבחוץ כגון האקרים או ספקית האינטרנט משום שיש לרשת הציבורית גישה רק לנתב מפני שהוא היחיד בעל כתובת ציבורית.



במסגרת העבודה שלי עשיתי שימוש במספר פרוטוקולים שונים:

- פרוטוקול ARP – הוא פרוטוקול תקשורת המשמש ברשת מחשבים לאיתור כתובת ה-MAC של מחשב ברשת על פי כתובת ה-IP שלה. פרוטוקול זה שיך לשכבת הקשר. הפרוטוקול מופעל בעבודה שלי כאשר אני יוצר מחדש את מנות המידע ויוצר את ה-frame לצורך שליחתה. לאחר מכן, אני שומר את ההתאמה בין כתובת IP מסוימת לכתובת MAC במבנה נתונים של מטמון ARP כדי שלא אצטרך לחזור על הפעולה פעם חוזרת לאותו מחשב.
- פרוטוקול ICMP – זהו פרוטוקול רשת בשכבת התעבורה שמשמש לצורך העברת שגיאות ומידע אודות פעולת מערכת מסוימת ברשת האינטרנט (למשל, כאשר חבילה אינה מגיעה ליעד שלה לפני שמספר הקפיצות שלה (TTL) מתאפס, נשלחת מנת ICMP על מנת ליידע זאת). נעשה שימוש בפרוטוקול זה בפעולת PING ו-TRACEROUTE.
- פרוטוקול TCP – זהו פרוטוקול הפועל בשכבות התעבורה ומבטיח העברה אמינה של נתונים בין שתי תחנות ברשת מחשבים באמצעות יצירת חיבור כאשר הוא עושה הפרוטוקול שימוש בפרוטוקול ה-IP לצורך העברת הנתונים. TCP מעביר את הנתונים, מוודא את נכונותם, ומאשר את קבלת הנתונים במלואם או מבקש שליחה מחדש של נתונים שלא הגיעו בצורה תקינה.
- פרוטוקול UDP – זהו פרוטוקול הפועל בשכבת התעבורה כאשר בניגוד ל-TCP הוא אינו מספק אמינות או שימור סדר כפי שקורה. חבילות מידע עשויות להגיע בסדר שונה מזה שבו הן נשלחו, להגיע מספר פעמים או ללכת לאיבוד ולא להגיע כלל. חסכון המידע המבטיח את אמינות השליחה ועובדת היותו של הפרוטוקול חסר קישור. גורמים לפרוטוקול UDP להיות מהיר בשליחה. המהירות היחסית של הפרוטוקול הופכת אותו מתאים ביותר לאפליקציות שאינן דורשות אמינות מלאה של המידע ולאפליקציות עבורן יש חשיבות גבוהה לזמן דיוור קצר כמו DNS.
- בחרתי לממש את שלושת הפרוטוקולים האלה מפני שהם הנפוצים ביותר בשימושם תחת חבילת הפרוטוקולים של TCP/IP.

במהלך כתיבת הפרויקט שילבתי מספר רב של נושאים וניסיתי שיהיה רב-תחומי ככל הניתן. ראשית, במהלך ההרצות ובניית סביבת העבודה, עבדתי לגמרי במערכת הפעלה LINUX בהפצת CentOs 7. כמו כן, הייתי צריך ללמוד לעומק על כל פרוטוקול בו השתמשתי על מנת שאוכל לקרוא כל חלק ממנת מידע שאקבל ולכתוב מחדש את מנת המידע החדשה שאשלח. יתרה מכך, עשיתי שימוש במודולים חיצוניים נפוצים כמו Scapy לשם הבנייה הסופית של מנת המידע הנשלחת ושליחתה בשכבת הרשת ומודול ה-WX לשם בניית ממשק גרפי (GUI) כדי לאפשר הפעלה נוחה של הפרויקט במחשב הנתב.



תיאור המוצר המוגמר

במסגרת פרויקט זה מימשתי מערכת NAT מבחינת תוכנה על גבי מערכת ההפעלה (לא פיתחתי דרייברים משלי). הרעיון מאחורי הפרויקט הוא כי כאשר התוכנה מורצת, כל מנות המידע אשר מגיעות למחשב בו היא רצה (מחשב הנתב) בדרכן לרשת החיצונית, מועבדות על מנת שיוכלו להישלח לרשת החיצונית באמצעות כתובתו החיצונית של הנתב והמידע אודות מנת המידע הנשלחת נשמר במבנה נתונים לפי הפרוטוקול של מנת המידע, כאשר התעסקתי עם הפרוטוקולים הנפוצים ביותר – TCP, ICMP ו-UDP. בכך כאשר מגיע התגובה למחשב הנתב, הוא יודע לאיזה מחשב ברשת הפרטית להחזיר את התגובה.

בעת ביצוע הפרויקט היו לי מספר מטרות לגבי המוצר הסופי:

- המטרה הראשונה והחשובה ביותר הייתה שהוא יענה על הצורך הבסיסי שגרם לפיתוח מערכות NAT וזהו היכולת להשתמש בכתובת ציבורית אחת בלבד על מנת לחבר מספר רב של מחשבים לרשת הציבורית כאשר כל הצדדים בשתי הרשתות יהיו "עיוורים" לקיומה ולא יתחשבו בה כלל.
- שהמוצר הסופי יהיה יעיל ולא ידרוש משאבים רבים על מנת שיוכל לרוץ במכונת נתב ביתית שהיא קטנה ולא בולטת בדרך כלל.
- שהממשק הגרפי יהיה נוח וממנו ניתן יהיה לדלות מידע אודות מעבר הנתונים מהרשת הפנימית לחיצונית ולהיפך ולצד זה שתהיה מערכת מעמיקה של קבצי log לפי רמת ההרצה של המערכת כך שיהיה גישה למידע מפורט ומעמיק אודות פעולתה וניתוחו.
- לאתגר את עצמי ולהרחיב את אופקי המידע שברשותי על נושאים שמגיל צעיר התעניינתי בהם ורציתי לדעת יותר עליהם ובכך ליהנות מאוד מכל התהליך, אפילו יותר מהנאתי מן התוצאה.
- ללמוד כמה שיותר על נושאים כגון – מערכות הפעלה מסוג LINUX, רשתות מידע ופיתוח בשפת פייתון. כל אלה הינם נושאים חדשים שלא למדתי במסגרת לימודית אלא רק בכוחות עצמי ולכן ביצעתי את הפרויקט בצורה עצמאית וללא מנחים.

זאת ועוד, המוצר המוגמר ענה על מספר רב של דרישות. ראשית, הוא אכן ביצע את מטרותיו ואפשר חיבור של מספר רב מאוד של מחשבים לרשת החיצונים ולשרתים מבחוץ, כאשר ההגבלה היחידה שחלה עליו הינה כמות הכתובות שניתן להקצות למחשבים ברשת הפנימית תחת ה-subnet של ממשק הרשת של הנתב ברשת הפנימית. הגבלה זו אפשרה חיבור מחשבים רבים. כמו כן, התוכנה רצה על מחשב בעל משאבים מאוד מוגבלים ביעילות ובמהירות, דבר שאידיאלי בהרצת נתב.

המערכת שמימשת עונה על השאלות הבאות:

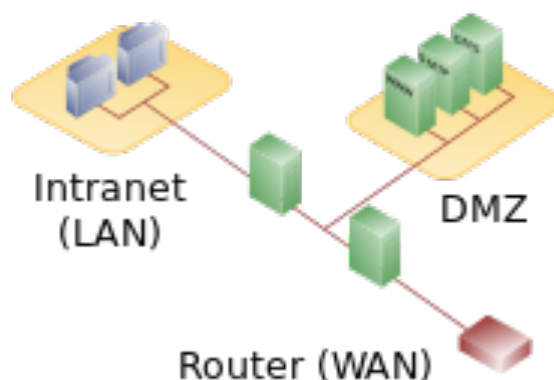
- ❖ **כיצד ניתן להקצאות בפועל כתובת ציבורית אחת למספר רב של מחשבים ברשת פרטית?** באמצעות המרת הכתובות במנות המידע המגיעות למחשב ה-NAT לכתובת הציבורית של מחשב זה ושמירת שיוך מנות המידע בטבלת ה-NAT.
- ❖ **איך ניתן לראות את כל התעבורה ברשת הפרטית אל הרשת החיצונית?** סיפקתי שתי אפשרויות לראיית התעבורה ברשת הפרטית אל החיצונית כאשר אחת היא נוחה יותר אך פחות מעמיקה (ממשק גרפי שיצרתי) ואילו השנייה היא הרצת המערכת ברמת הרצה של Debug, דבר השולח את כל פעולות הרשת לקובץ log מיועד ומסודר.
- ❖ **כיצד נתבים ביתיים הנמצאים כמעט בכל בית בימינו מאפשרים את מעבר הנתונים לרשת החיצונית כך שגם מחשבי הרשת החיצונים וגם מחשבי הרשת הפרטים "עיוורים" להמרה שהיא מבצעת?** מערכת ה-NAT מותאמת לכך שהיא מטפלת באופן עצמאי בכל החבילות המנותבות דרכה לרשת אחרת, היא מאחסנת את אופן טיפולם ובוחרת איך לטפל במנות המידע. דבר הלא מצריך התייחסות מצד הצדדים האחרים.

מבט אישי על תהליך הפיתוח

בתהליך העבודה על הפרויקט, היו לי מספר אתגרים. ראשית, כל ילדי כיתתי אמורים לבצע פרויקט שונה, אך מגיל צעיר ידעתי שארצה ללמוד על רשתות מחשבים וראיתי בפרויקט זה הזדמנות מעולה לכך ולכן ביקשתי אישור מיוחד ללמוד את כל החומר הן על רשתות והן על כתיבה בפייתון בצורה עצמאית. אחד הדברים האהובים עליי זה ללמוד בצורה עצמאית ולחקור ולדרוש בכל נושא בו אני נתקל, לכן העובדה כי לא היה לי מורה או מנחה מסוים לא היוותה מכשול. לפיכך, בתחילת השנה התחלתי ללמוד פייתון ורשתות מהספרים שבאתר cyber.org.il ולאחר מכן עברתי לביצוע הפרויקט תוך כדי למידת כל נושא בדרך והרחבת הידע של בצורה משמעותית.

כאשר הייתי בראשית דרכי בכתיבת העבודה, הייתי צריך ללמוד את הפרוטוקולים עימם אני אתעסק בחלק המרכזי של עבודתי – TCP, ICMP ו-UDP. למדתי את החלקים של כל מנת מידע מכל אחד מהפרוטוקולים המדוברים לעומק, תפקידם, גודלם והשימוש בהם. כמו כן, הייתי צריך להבין את כותרת ה-IP במנת המידע, מהו struct ואיך משתמשים בו.

כמו כן, למדתי כי נהוג להבדיל בין שני סוגי NAT כלליים. הסוג הראשון שבדרך כלל נקרא NAT או NATP מתייחס לתרגום כתובות המלווה בתרגום הפורטים וטכניקה זו מאפשרת, לדוגמה, לחבר מספר גדול של מחשבים לרשת האינטרנט באמצעות כתובת IP ציבורית אחת. הסוג השני נקרא לפעמים NAT סטטי שמתייחס לתרגום כתובת ה-IP בלבד, ולכן דורש כתובת ציבורית אחת עבור כל חיבור. טכניקה זו נקראת גם "אזור מפורז – DMZ". היא מאפשרת למחשב לקבל כל חיבור חיצוני על אף שכתובת ה-IP נמצאת בשימוש על ידי הנתב עצמו. NAT שכולל תרגום פורטים גם הוא נבדל לשני סוגים: תרגום כתובות מקור ותרגום כתובות יעד.

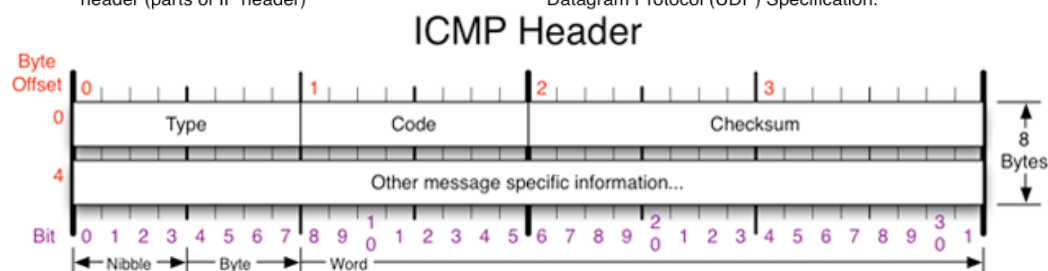
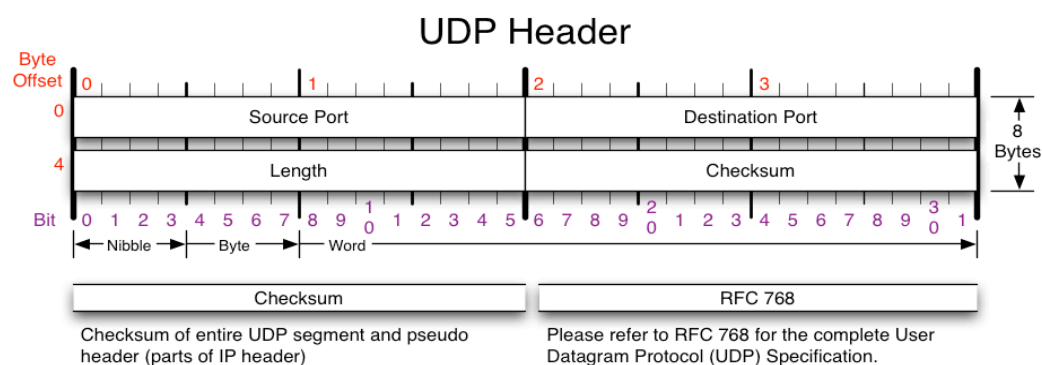
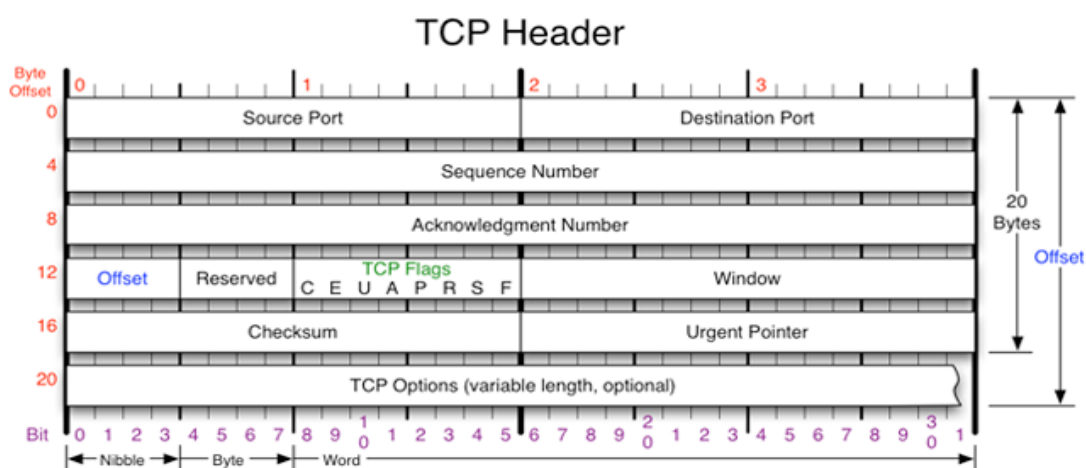


בעבודתי ללא כל התלבטות בחרתי לממש NAT המתרגם את הפורטים ואת כתובות המקור מפני שזהו הסוג הנפוץ ביותר ברוב הבתים בימינו והצורך לחיבור מספר מחשבים רבים רווח יותר בקרב רוב האנשים מאשר הצורך לבטיחות ופרטיות מוגברת עליה עונה NAT המספק DMZ.

לאחר מכן מימשתי חלק זה של בניית ה-NAT, זהו החלק העיקרי בפרויקט ועל מנת שלא יהיו עיכובים וההפעלה של קליטת ושליחת מנות מידע מכל אחת משני ממשקי הרשת של המחשב, בניתי תהליכון שונים (Thread) לכל אחת מארבעת מטרות אלה וככה הקליטה והשליחה לא תלויה באף דבר אחר המתנהל בתוכנה.

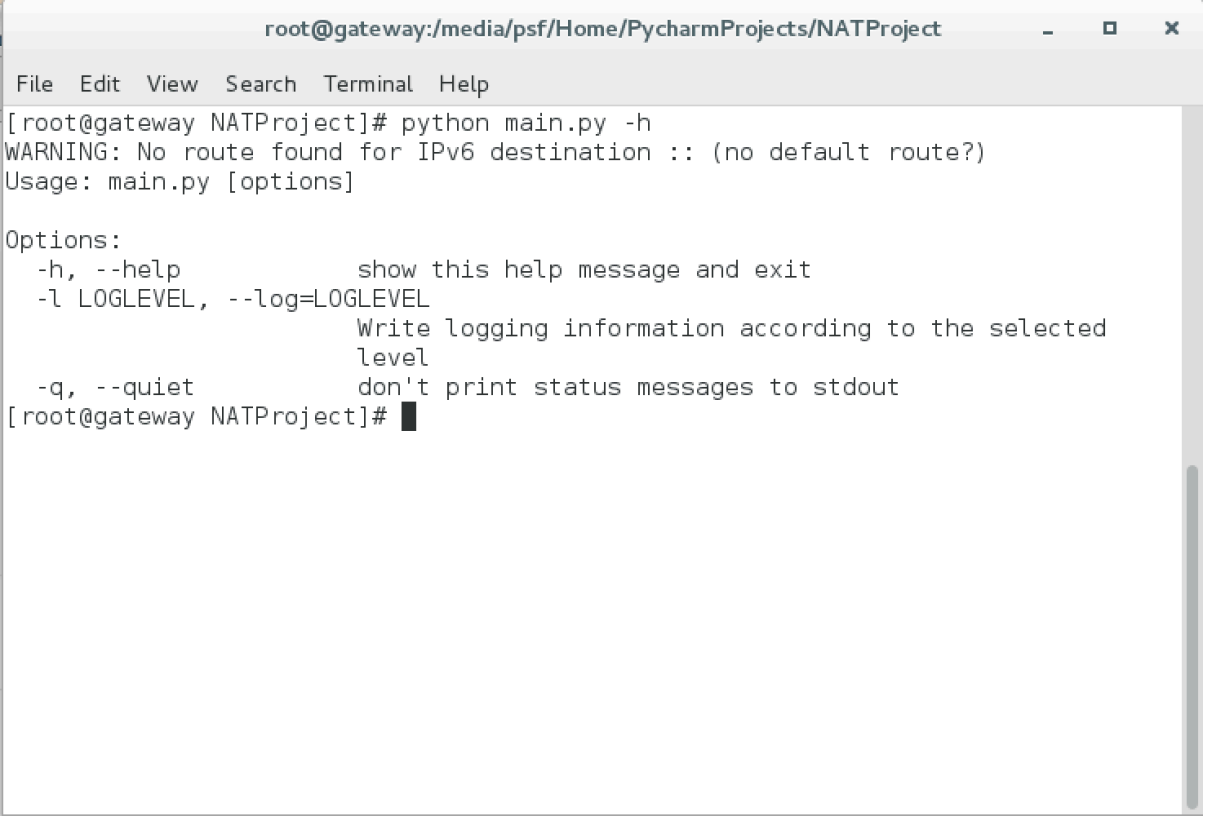
זאת ועוד, לאחר מימוש ה-NAT, לצורך הרצה בתנאים המתאימים ביותר ומעקב אחרי כל הרצה לפי סוג המעקב המבוקש. הוספתי options בעת הפעלת התוכנה מן ה-console שמורות איזה סוג הרצה מעוניינים ולפי ההרצה שנבחרה נפלט קובץ log בעל תוכן שונה.

לאחר מימוש פן זה בעבודתי, עברתי לבניית ממשק גרפי ונוח המאפשר לחזות בפעולות מערכת ה-NAT. ממשק זה מראה את עשרת מנות המידע השונות שהגיעו ונשלחו וכל המידע עליהן.



תיאור הממשק למשתמש

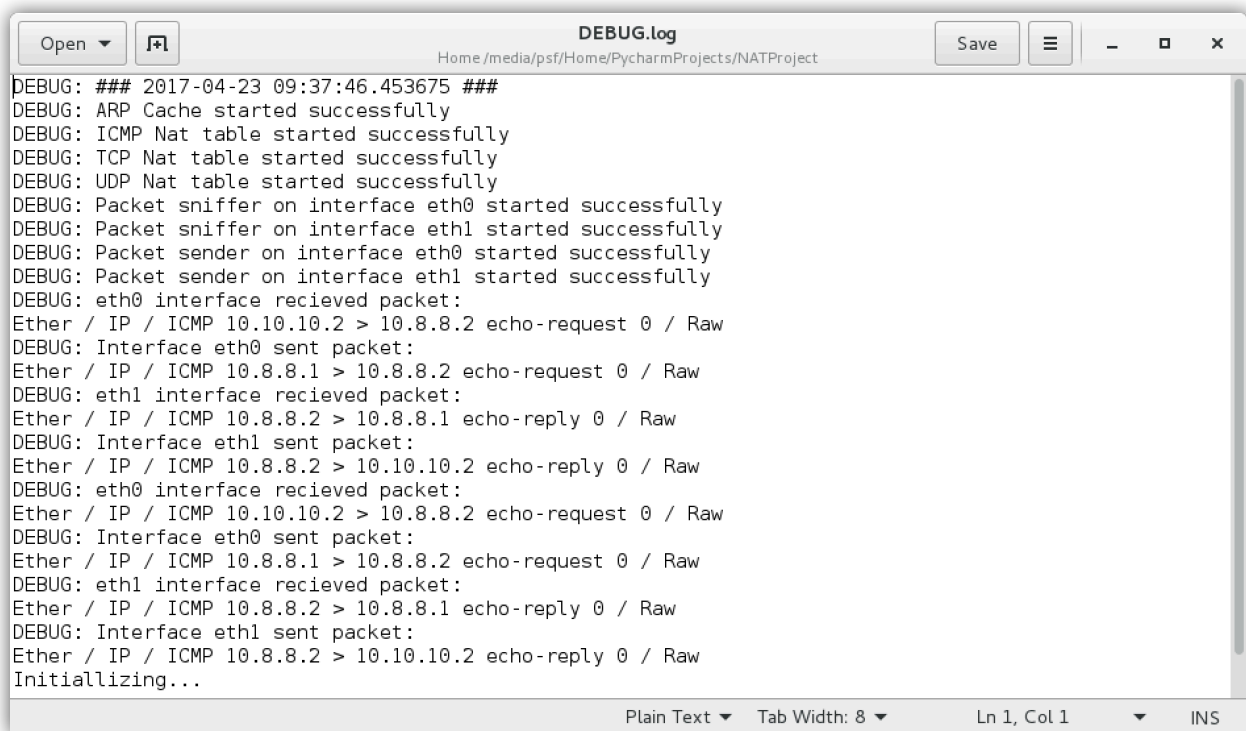
ראשית, חשוב להדגיש כי המערכת רצה לחלוטין ברקע וכלל אינה דורשת התערבות של המשתמש. הפעלת התוכנה מתרחשת דרך ה-Terminal בהרשאות גבוהות. ניתנת האפשרות להיכנס למצב הרצה מסוים בו יפלט לקובץ log מידע על פי המצב הנבחר וזה באמצעות האופציה: `-l/--log`. גם ניתן להפעיל את התוכנה במצב שקט, כלומר ללא כל פלט ל-Terminal באמצעות: `-q/--quiet`.



```
root@gateway:/media/psf/Home/PycharmProjects/NATProject
File Edit View Search Terminal Help
[root@gateway NATProject]# python main.py -h
WARNING: No route found for IPv6 destination :: (no default route?)
Usage: main.py [options]

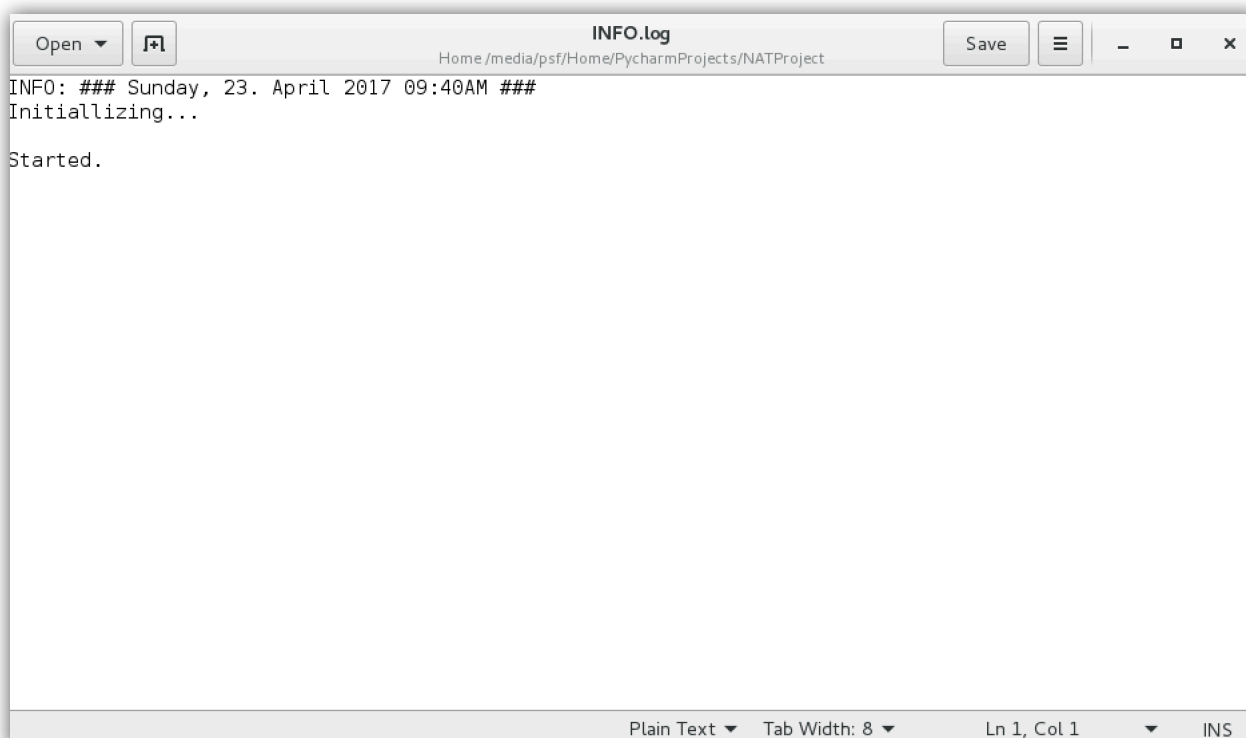
Options:
  -h, --help            show this help message and exit
  -l LOGLEVEL, --log=LOGLEVEL
                        Write logging information according to the selected
                        level
  -q, --quiet            don't print status messages to stdout
[root@gateway NATProject]#
```

- ישנן 3 רמות שונות של הרצה עם קבצי לוג –
- Debug – מידע מפורט על כל מה שמתרחש בתוכנית: מעבר מנות מידע, התחלת חלקים שונים, בעיות, הדפסות וכולי.
 - Info – ההדפסה עוברת מהדפסה ל-Terminal להדפסה לקובץ log.
 - Error – כל השגיאות אותן מביאה המערכת, שגיאות בהרצת התוכנית או שגיאות בטיפול בפאקטות מאוחסנות בקובץ log זה.



DEBUG: ### 2017-04-23 09:37:46.453675 ###
DEBUG: ARP Cache started successfully
DEBUG: ICMP Nat table started successfully
DEBUG: TCP Nat table started successfully
DEBUG: UDP Nat table started successfully
DEBUG: Packet sniffer on interface eth0 started successfully
DEBUG: Packet sniffer on interface eth1 started successfully
DEBUG: Packet sender on interface eth0 started successfully
DEBUG: Packet sender on interface eth1 started successfully
DEBUG: eth0 interface recieved packet:
Ether / IP / ICMP 10.10.10.2 > 10.8.8.2 echo-request 0 / Raw
DEBUG: Interface eth0 sent packet:
Ether / IP / ICMP 10.8.8.1 > 10.8.8.2 echo-request 0 / Raw
DEBUG: eth1 interface recieved packet:
Ether / IP / ICMP 10.8.8.2 > 10.8.8.1 echo-reply 0 / Raw
DEBUG: Interface eth1 sent packet:
Ether / IP / ICMP 10.8.8.2 > 10.10.10.2 echo-reply 0 / Raw
DEBUG: eth0 interface recieved packet:
Ether / IP / ICMP 10.10.10.2 > 10.8.8.2 echo-request 0 / Raw
DEBUG: Interface eth0 sent packet:
Ether / IP / ICMP 10.8.8.1 > 10.8.8.2 echo-request 0 / Raw
DEBUG: eth1 interface recieved packet:
Ether / IP / ICMP 10.8.8.2 > 10.8.8.1 echo-reply 0 / Raw
DEBUG: Interface eth1 sent packet:
Ether / IP / ICMP 10.8.8.2 > 10.10.10.2 echo-reply 0 / Raw
Initiallizing...

Plain Text ▾ Tab Width: 8 ▾ Ln 1, Col 1 ▾ INS



INFO: ### Sunday, 23. April 2017 09:40AM ###
Initiallizing...

Started.

Plain Text ▾ Tab Width: 8 ▾ Ln 1, Col 1 ▾ INS

כמו כן, לאחר הרצה הפרויקט מוצג ממשק גרפי ממנו ניתן לראות את עשרת מנות המידע הנכנסות למחשב ה-NAT וכל המידע עליהן ואת עשרת מנות המידע היוצאות ממחשב ה-NAT וכל המידע עליהן. הדבר מאפשר לחזות בהמרה של הכתובות ובפעולות ה-NAT ולעקוב אחר תעבורת המידע מן הרשת הפנימית לרשת החיצונית.

NATmonitor						
	Type	Source IP	Source Port	Destination Ip	Destination Port	Time
Incoming:						
1						
2						
3						
4						
5						
6						
Outgoing:						
1						
2						
3						
4						
5						
6						

7						
8						
9						
10						

NATmonitor						
	Type	Source IP	Source Port	Destination Ip	Destination Port	Time
Incoming:						
1	ICMP	10.8.8.2	-	10.8.8.1	-	01:44:51PM
2	ICMP	10.10.10.2	-	10.8.8.2	-	01:44:50PM
3	UDP	10.8.8.2	53	10.8.8.1	1025	01:44:41PM
4	UDP	10.10.10.3	59177	10.8.8.2	53	01:44:40PM
5						
6						
Outgoing:						
1	ICMP	10.8.8.2	-	10.10.10.2	-	01:44:52PM
2	ICMP	10.8.8.1	-	10.8.8.2	-	01:44:50PM
3	UDP	10.8.8.2	53	10.10.10.3	59177	01:44:42PM
4	UDP	10.8.8.1	1025	10.8.8.2	53	01:44:41PM
5						
6						

זאת ועוד, כתבתי קובץ סקריפט שמתחיל את האפליקציה בצורה אלגנטית עם כל מרכיביה:

```
#!/bin/bash
```

```
trap "" HUP
```

```
iptables -A FORWARD -j DROP
```

```
iptables -A OUTPUT -p tcp -s 10.8.8.1 -d 10.8.8.2 --dport 80 -j DROP
```

```
/sbin/service iptables save > /dev/null
```

```
systemctl restart iptables
```

```
trap - HUP
```

```
echo -e "Enter NAT program parameters:"
```

```
read parameters
```

```
python Main.py $parameters
```

```
[parallels@gateway NATProject]$ sudo sh NATscript.sh
Enter NAT program parameters:
-l ERROR
(You can always type EXIT to close application)
Initialllizing...

Started.
█
```

סביבת העבודה

את קוד הפרויקט רשמתי לחלוטין ב-Python 2.7.10. הסיבות לכך הן פשטות ונוחות השפה, העובדה שאני מגיע מרקע של תכנות מונחה עצמים ורצוני ללמד את עצמי לתכנת בשפה זו. שמעתי רבות על נוחות השפה ויעילותה וכמה היא נפוצה בימינו ולכן היא תמיד סקרנה אותי ורציתי ללמוד אותה. סביבת העבודה בה השתמשתי עבור שלוש השפות הינה PyCharm. זוהי תוכנה מקצועית ונוחה לעבודה על פרויקטים מרובי קבצים ומאפשרת גם הרצה ו-debugging של הסקריפטים.



יתרה מכך, לצורך ההרצה עצמה דרוש לי מספר רב של מחשבים וזה מפני שאני צריך מחשב שישמש כנתב, מחשב שיש כשרת ולפחות שני מחשבים שישמשו כקליינטים אשר יוכיחו כי מערכת ה-NAT שלי מגשימה את מטרתה ומאפשרת שימוש בכתובת אחת ציבורית לרשת פרטית של מחשבים בגישתם אליה. ידוע לי כי הרוב המוחלט של השרתים בשימוש משתמשים במערכות מבוססות LINUX ומסיבה זו ועל מנת שאתנסה יותר בשימוש במערכות הפעלה מסוג זה וב-Terminal שלהן – כל ארבעת המחשבים היו למעשה מכונות וירטואליות ברשת סגורה המריצים CentOs 7.

הגדרתי את ממשקי הרשת של כל אחד מהם בהתאם כדי שהמחשבים ברשת הפרטית תהיה כתובת של הרשת הפרטית, לשרת יהיה כתובת של רשת שונה ("הציבורית") ואילו למחשב הנתב יהיו שני ממשקים לכל אחד מהרשתות המדוברות.

על מנת להריץ את השרת כדי להדגים את הפרוטוקולים של TCP ו-UDP התקנתי Apache HTTP Server והטענתי אליו קובץ HTML פשוט שכתבתי ו-Bind כדי להריץ שרת DNS שיוכל להחזיר את כתובת השרת.

השתמשתי במהלך הפרויקט שלי בשני מודולים חיצוניים:

❖ Scapy – זהו מודול לבניית מנות מידע ושליחתן. נוח מאוד לבנות לאחר עריכת מנת המידע את מנת המידע שתוכנת ה-NAT תשלח באמצעות Scapy. כמו כן, Scapy מציע פונקציות להצגת מנות מידע על פי השכבות שלהן וניתן ללמוד ממנו רבות על חלקי מנות המידע והשימוש בהן.

❖ WxPython – זהו מודול המשמש ככלי ליצירת ממשקים גרפיים הרצים במערכות רבות. באמצעות מודול זה יצרתי את הממשק הגרפי של המערכת שלי ובה הצגתי את מעבר הפאקטות וניתובן דרך תוכנת ה-NAT.

* חשוב לי לציין שבגרסת התוכנה בה התבקשתי לכתוב את הממשק הגרפי של (WxPython) למערכת ההפעלה Centos בה הרצתי את ה-NAT יש באג ידוע ומוכר בקהילת המפתחים בו כאשר מדי פעם הממשק קורס ומפסיק לפעול. פלט השגיאה כתוצאה מהבאג הינו:

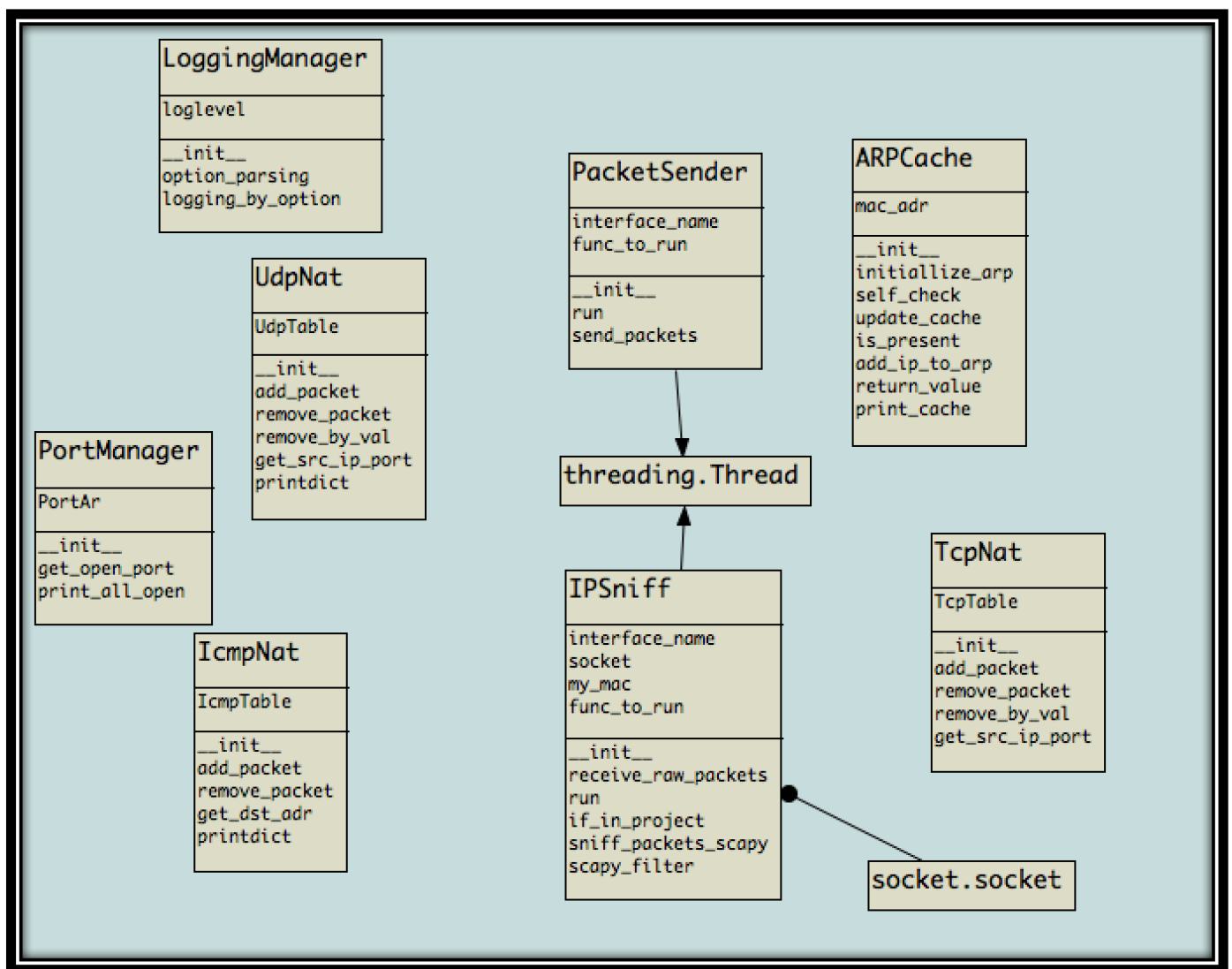
```
Gdk:ERROR:gdkregion-generic.c:1201:miUnionO: assertion failed:  
(pReg->numRects<=pReg->size)
```



תיאור המודולים

הפרויקט שלי מורכב ממספר מודולים שונים:

- מודול IpHandler – מודול זה כולל את כל הפעולות המחזירות ועורכות את ערכי חלק ה-IP בפאקטה. מודול זה עושה שימוש במודולים של socket, struct ו-fcntl.
- מודול IcmpHandler – מודול זה כולל את כל הפעולות המחזירות ועורכות את ערכי חלק ה-ICMP בפאקטת ICMP.
- מודול TcpHandler – מודול זה כולל את כל הפעולות המחזירות ועורכות את ערכי חלק ה-TCP בפאקטת TCP.
- מודול UdpHandler – מודול זה כולל את כל הפעולות המחזירות ועורכות את חלק ה-UDP בפאקטת UDP.
- מודול SendReceive – מודול זה כולל את כלל הפעולות לשליחת פאקטות מפרוטוקול TCP, ICMP ו-UDP. כמו כן, הוא עושה זאת באמצעות ייבוא המודולים של IpHandler, IcmpHandler, TcpHandler ו-Udphandler.
- מודול LoggingSystem – מודול זה אחראי על המערכת ליצירת קבצי לוג על פי רמת ההרצה שנבחרה על ידי המשתמש, היא כוללת את מבנה הנתונים של LogManager שאחראי על כך ועל עיבוד הפרמטרים שהתקבלו בעת הפעלת התוכנה.
- מודול NATmonitor – מודול זה אחראי על הצגת הממשק הגרפי וניהולו והוא כולל מבנה נתונים מתאים. הוא מייבא לשם כך את המודול החיצוני של WxPython.
- מודול Main – מודול זה הוא העיקרי בפרויקט ומנהל ודואג לעבודתם של כל שאר חלקי הפרויקט. הוא מפעיל תהליכונים (Thread) לקליטת ושליחת פאקטות, מאתחל את טבלאות ה-Nat ואת מטמון ה-ARP, דואג לתפעול מערכת קבצי הלוג ומתחיל את הממשק הגרפי. הוא מייבא את מודולים NATmonitor, LoggingSystem ו-SendRecieve.



תיעוד הקוד ואלגוריתמים

יש לי מספר מבני נתונים ומחלקות בקבצי הפרויקט:

- מחלקת IpSniff: מבנה נתונים זה אחראי על קליטת הפקטות מממשק רשת מסוים כתהליכון (Thread). יצירת עצם שכזה מקבל כפרמטר את שם הממשק ממנו רוצים להתחיל לקלוט פאקטות ואת הפונקציה בה רוצים להשתמש על מנת להסניף אותן. יש שתי פעולות כאשר האחת משתמש במודול ה-socket והיא ישירה ויעילה ואילו השנייה משתמשת במודול scapy כאשר היא פשוטה, קצרה אך איטית ולא יעילה, לפיכך בחרתי להשתמש בפעולה הראשונה.
ראשית, פעולה זו רצה כלולאה תמידית כחלק מהתהליכון, במידה ומגיעה מנת מידע למחשב היא בודקת שהפאקטה לא ריקה ובמידה והיא ריקה זורקת אותה. אחרת, הפעולה משייכת את הפאקטה לפי הממשק רשת ממנו היא הגיעה ובודקת האם יש למערכת את כתובות מחשב המוצא של הפאקטה במטמון ה-ARP ובמידה ואין לה, היא מוסיפה אותם. לאחר מכן, היא מוסיפה את הפאקטה לתור שיצרתי לכל ממשק רשת המיועד לפאקטות המיועדות לטיפול.
- מחלקת PacketSender: מחלקה זו אחראית על שליחת פאקטות מהממשק שמורים עליה כתהליכון רקע. הפעולה אשר רצה ברקע פועלת כך: היא מסווגת את הממשק רשת ממנו היא אמורה לשלוח פאקטות לפי הפרמטר שהעצם קיבל. לאחר מכן, היא בודקת אם התור של כל ממשק רשת של פאקטות המיועדות לטיפול ריק ובמידה ולא היא מושכת מתור זה את הפאקטה.
הפעולה שומרת את נתוני הפאקטה בטבלת ה-NAT המיועדת לה לפי פרוטוקול הפאקטה על מנת שנוכל כאשר תחזור התגובה לדעת לאיזה מחשב ברשת הפרטית להחזיר את הפאקטה. הפעולה קוראת לפעולה אחרת שמשנה את הפאקטה בהתאם לסוגה ומחליפה את הכתובת מקור והפורט שלה כך שנוכל לשלוח אותה. לבסוף, הפעולה במחלקה המדוברת שולחת את הפאקטה ששונתה על פי הפרוטוקול שלה.

- מחלקת ARP-Cache: מחלקה זו אחראית על מטמון ה-ARP של המערכת שלי והיא מאחסנת מילון של כתובת ה-IP וכתובת ה-mac, כאשר מפתחות המילון הינם כתובות ה-IP. נוספים למילון זה ערכים במספר דרכים. הראשונה, על ידי קבלת פאקטה מכתובת שלא במילון ואז היא מוספת בצורה אוטומטית למטמון. השנייה, באתחול המחלקה, נערכת סריקה של התת רשתות של רשתות הנתב למחשבים והם מוספים למחלקה זו.
- כמו כן, במחלקה זו ישנן פעולות לאחזור כתובת mac או IP על פי השנייה, פעולה להוספת ערך למילון ופעולה להדפסת המילון.
- מחלקת IcmpNat: מחלקה זו מארגנת את נתוני הפאקטות ש"תורגמו" ושונו כתובתם במסגרת ה-NAT תחת פרוטוקול ICMP. מבנה נתונים זה מורכב ממילון כאשר מפתחות המילון הינם ה-ICMP ID, מזהה ייחודי לפאקטת ICMP (שנוצר בתחילה באופן רנדואלי) וערכי המילון הינם כתובות IP המקור של הפאקטות.
- כאשר מגיעה ל-NAT פאקטת ICMP, מוספים למילון המספר זיהוי שלה וכתובת המקור שלה, הפאקטה משונה ונשלחת ליעד. כאשר בתגובה חוזר ל-NAT פאקטת ICMP, נערך חיפוש במילון לפי מספר הזיהוי וכאשר הוא נמצא, הפאקטה משונה על פי כתובת המקור הישן (הופכת לכתובת היעד החדשה) והערך נמחק מהמילון.
- כמו כן, במחלקה זו ישנן פעולות להדפסת המילון, הוספה והוצאת ערכים וקבלת כתובת המקור לפי מספר הזיהוי.
- מחלקת TcpNat: מחלקה המארגנת את נתוני הפאקטות ש"תורגמו" ושונו כתובתם והפורטים במסגרת ה-NAT תחת פרוטוקול TCP. מבנה נתונים זה מורכב ממילון כאשר מפתחות המילון הינם פורטי היציאה של הפאקטות ממחשב הנתב. מזהה זה ניתן על ידי כאשר אני עורך את הפאקטה וערכי המילון הינם כתובות IP המקור של הפאקטות ופורטי המקור של הפאקטות (כ-tuple).
- כאשר מגיעה ל-NAT פאקטת TCP, מוספים למילון המספר הפורט ממנה יצאה מה-NAT ו-tuple של כתובת המקור שלה ופורט המקור שלה, הפאקטה משונה ונשלחת ליעד. כאשר בתגובה חוזר ל-NAT פאקטת TCP, נערך חיפוש במילון לפי הפורט אליו הגיע במכונת הנתב, הפאקטה משונה על פי כתובת המקור הישן (הופכת לכתובת היעד החדשה) והפורט הישן והערך נמחק מהמילון.
- כמו כן, במחלקה זו ישנן פעולות להדפסת המילון, הוספה והוצאת ערכים וקבלת כתובת המקור והפורט לפי פורט היציאה.

- מחלקת UdpNat: מחלקה העובדת בצורה זהה למחלקת TcpNat. מילון זהה, פעולות זהות ודרך פעולה זהה.
- מחלקת LoggingManager: מחלקה זו אחראית על פעולת מערכת קבצי הלוג במידה ובעת הרצת התוכנית מתקבל פרמטר של קבצי לוג. מחלקה זו כוללת שתי פעולות: הראשונה אחראית על הפעלת אופציות הפרמטרים בעת הרצת התוכנית (ישנן מספר אופציות לפרמטרים – מערכת קבצי הלוג, לחסום הדפסה ועזרה (-h)). לעומת זאת, השנייה מתעסקת אך ורק בהפעלת רמות ההרצה השונות של התוכנית. הראשונה היא Debug המציעה את כל פעולות התוכנית בפירוט (החל מעליית כל מרכיב ועד להצגת כל פאקטה המגיעה ונשלחת), Info המעביר את ההדפסה מהקונסול לקובץ לוג ו-Error, המעתיקה את כל השגיאות במהלך ההרצה לקובץ לוג.
- מחלקת NATmonitor: מחלקה זו אחראית על הפעלת הממשק הגרפי והצגת כל העברות מנות המידע דרך התוכנית. בממשק הגרפי ישנה טבלה הכוללת 6 שורות לפאקטות יוצאות ו-6 שורות לפאקטות נכנסות כאשר על כל פקטה שמועברת נכתב סוגה, כתובת מקורה, פורט מקורה, כתובת יעד, פורט יעד וזמן ההעברה. מחלקה זו מורכבת ממספר פעולות: הראשונה שמופעלת הינה פעולת האתחול שפותחת את חלון הממשק, מסדרת את הטבלה ונתוניה ומחכה להגעת פאקטות. לצורך בניית הפרויקט בצורה מודולרית ככל שניתן, מועברות פאקטות להצגה בממשק באמצעות באפר המשותף לתוכנית הראשית אליו הוא שולח פאקטות. כמו כן, עצם הממשק מקבל אובייקט Thread וזה משום שאני מפעיל אותו כהליך בפני עצמו על מנת שלא יגרום לקטיעות ובעיות בהרצת שאר חלקי התוכנית. כאשר מופעל אובייקט מסוג מחלקה זו, הוא פועל ברקע וכאשר נכנסות או יוצאות פאקטות מה-NAT הוא ישר מציג אותן בממשק אחד אחרי השני, כאשר הרשומה האחרונה היא הראשונה בטבלת הממשק.

תדפיס הקוד

MAIN.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: Main                                                #
# Version: 1.0                                              #
# Linux Tested Versions: Centos 7 64 bit                    #
# Python Tested Versions: 2.7.10 32-bit                     #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

import socket
import os
import Queue
import time
from subprocess import PIPE, Popen
import threading
from LoggingSystem import *
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
from SendRecieve import *
from NATmonitor import *

class ARPCache: # Responsible for starting the ARP Cache and Managing
it.

    def __init__(self):
        self.mac_adr = {}
        self.self_check()
        self.initiallize_arp()

    def initiallize_arp(self):
        print "Initialllizing...\n"
        for ip in range(1, 3):
            for subnet in range(0, 2):
                if subnet == 0:
                    process = subprocess.Popen(["ping", "-c", "1",
"10.10.10." + str(ip)], stdout=subprocess.PIPE)
                    process.wait()
                else:
                    process = subprocess.Popen(["ping", "-c", "1",
"10.8.8." + str(ip)], stdout=subprocess.PIPE)
                    process.wait()
```

```

    print "Started."
    logging.debug('ARP Cache started successfully')

    def self_check(self): # Loads the values of the two web interfaces
to the ARP Cache
        self.mac_adr[get_inf_ip(InterfaceList[2])] =
get_inf_mac(InterfaceList[2])
        self.mac_adr[get_inf_ip(InterfaceList[1])] =
get_inf_mac(InterfaceList[1])

    def update_cache(self, ip, mac): # Updates ARP Cache with given
values
        self.mac_adr[ip] = mac

    def is_present(self, ip):
        if ip in self.mac_adr:
            return True
        else:
            return False

    def add_ip_to_arp(self, ip): # Add Value to ARP Cache by ip, works
by console
        pid = Popen(["arp", "-n", ip], stdout=PIPE)
        s = pid.communicate()[0]
        self.update_cache(ip, re.search(r"([a-f\d]{1,2}\:){5}[a-
f\d]{1,2})", s).groups()[0])

    def return_value(self, ip): # Returns the mac address with given
IP
        if ip in self.mac_adr:
            return self.mac_adr[ip]
        else:
            self.add_ip_to_arp(ip)
            return self.mac_adr[ip]

    def print_cache(self): # Prints the content of ARP Cache
Dictionary
        for key, value in self.mac_adr.items():
            print "-->" + key
            print value

class IPSniff(threading.Thread): # Responsible for sniffing packet
arriving to given interface
    def __init__(self, interface_name, sniff_func):
        threading.Thread.__init__(self)

        self.interface_name = interface_name
        self.socket = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.htons(ETH_P_IP))
        self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 2
** 30)
        self.socket.bind((self.interface_name, ETH_P_IP))

        self.my_mac = [get_if_hwaddr(i) for i in get_if_list()]
        self.func_to_run = sniff_func

```

```

def receive_raw_packets(self): # Recieves raw packets arriving at
the given interface using socket module
    while True:
        pkt, dummy = self.socket.recvfrom(MTU)
        if len(pkt) > 0:
            try:
                if self.interface_name == InterfaceList[2]:
                    if self.if_in_project(self.interface_name,
pkt):
                        logging.debug("%s interface recieved
packet: \n%s", self.interface_name, summary_dump(pkt))
                        gui_entry('Incoming', gui_parse(pkt))
                        if not
ARPCache.is_present(get_src_adr(pkt)):
                            ARPCache.update_cache(get_src_adr(pkt),
get_src_mac(pkt))
                        ArrivedPackets.put(pkt)
                else:
                    if self.if_in_project(self.interface_name,
pkt):
                        logging.debug("%s interface recieved
packet: \n%s", self.interface_name, summary_dump(pkt))
                        gui_entry('Incoming', gui_parse(pkt))
                        if not
ARPCache.is_present(get_src_adr(pkt)):
                            ARPCache.update_cache(get_src_adr(pkt),
get_src_mac(pkt))

                        ReturnedPackets.put(pkt)
            except Exception, message:
                log.error(message)

def run(self):
    getattr(self, self.func_to_run)()

def if_in_project(self, interface, pkt): # Checks if given IP is
within the test conditions of the project
    pkt = str(pkt)
    if interface == InterfaceList[2]:
        if get_src_adr(pkt)[:8] == '10.10.10' and \
(get_dst_adr(pkt)[:8] == '10.10.10' or
get_dst_adr(pkt)[:6] == '10.8.8'):
            return True
    elif interface == InterfaceList[1]:
        if get_src_adr(pkt)[:6] == '10.8.8' and \
(get_dst_adr(pkt)[:8] == '10.10.10' or
get_dst_adr(pkt)[:6] == '10.8.8'):
            return True
    return False

```

```

    def sniff_packets_scapy(self): # Scapy sniffing method: NOT
USED!
        sniff(iface="eth0", lfilter=self.scapy_filter(), prn=lambda x:
PacketList.append(x))

    def scapy_filter(self, pkt): # Scapy filter for scapy sniffer: NOT
USED!
        return pkt[Ether].src != [get_if_hwaddr(i) for i in
get_if_list()]

class PacketSender(threading.Thread): # Responsible for sending out
the waiting-in-line packets for given interface

    def __init__(self, interface_name):
        threading.Thread.__init__(self)
        self.interface_name = interface_name
        self.func_to_run = 'send_packets'

    def run(self):
        getattr(self, self.func_to_run)()

    def send_packets(self): # Sends out packets waiting in the given
interface
        if self.interface_name == InterfaceList[2]:
            while True:
                try:
                    if not ArrivedPackets.empty():
                        pkt = ArrivedPackets.get()
                        if ord(pkt[23]) == 1:
                            ICMPNat.add_packet(pkt)
                        pkt = packet_manipulator(pkt)
                        if pkt != '':
                            logging.debug('Interface %s sent packet:
\n%s', self.interface_name, summary_dump(pkt))
                            gui_entry('Outgoing', gui_parse(pkt))
                        if ord(pkt[23]) == 1:
                            send_icmp(pkt)
                        elif ord(pkt[23]) == 6:
                            send_tcp(pkt)
                        elif ord(pkt[23]) == 17:
                            send_udp(pkt)
                except Exception, message:
                    logging.error(message)

            elif self.interface_name == InterfaceList[1]:
                while True:
                    try:
                        if not ReturnedPackets.empty():
                            pkt = ReturnedPackets.get()
                            pkt = packet_manipulator(pkt)
                            if pkt != '':
                                logging.debug('Interface %s sent packet:
\n%s', self.interface_name, summary_dump(pkt))
                                gui_entry('Outgoing', gui_parse(pkt))
                                if ord(pkt[23]) == 1:
                                    send_icmp(pkt)

```

```

                                ICMPNat.remove_packet(pkt)  #Change
Location
                                elif ord(pkt[23]) == 6:
                                    send_tcp(pkt)
                                elif ord(pkt[23]) == 17:
                                    send_udp(pkt)
                                except Exception, message:
                                    logging.error(message)

class IcmpNat: # Stores dictionary for NAT Table for ICMP type packets
by storing ICMP ID and Address

    def __init__(self):
        self.IcmpTable = {}
        logging.debug('ICMP Nat table started successfully')

    def add_packet(self, pkt):
        if get_icmp_id(pkt) not in self.IcmpTable:
            self.IcmpTable[get_icmp_id(pkt)] = get_src_adr(pkt)

    def remove_packet(self, pkt):
        del self.IcmpTable[get_icmp_id(pkt)]

    def get_dst_adr(self, pkt):
        return self.IcmpTable.get(get_icmp_id(pkt))

    def printdict(self):
        for key, value in self.IcmpTable.items():
            print "-->" + str(key)
            print(value)

class TcpNat: # Stores dictionary for NAT Table for TCP type packets
by storing outgoing port and Source Address, port

    def __init__(self):
        self.TcpTable = {}
        logging.debug('TCP Nat table started successfully')

    def add_packet(self, pkt):
        for i in range(1, 55500):
            if i not in self.TcpTable or self.TcpTable[i] ==
(get_src_adr(pkt), get_src_port(pkt)):
                self.TcpTable[i] = (get_src_adr(pkt),
get_src_port(pkt))
                break
        return i + 1024

    def remove_packet(self, port):
        del self.TcpTable[port - 1024]

    def remove_by_val(self, SrIp, SrPrt):
        for i in range(0, 55500):
            if self.TcpTable.has_key(i):
                if self.TcpTable[i] == (SrIp, SrPrt):
                    del self.TcpTable[i]

```

```

def get_src_ip_port(self, port):
    if (port - 1024) in self.TcpTable:
        return self.TcpTable[port - 1024]
    return 0

class UdpNat: # Stores dictionary for NAT Table for UDP type packets
               by storing outgoing port and Source Address, port

    def __init__(self):
        self.UdpTable = {}
        logging.debug('UDP Nat table started successfully')

    def add_packet(self, pkt):
        for i in range(1, 55500):
            if i not in self.UdpTable or self.UdpTable[i] ==
(get_src_adr(pkt), get_src_port(pkt)):
                self.UdpTable[i] = (get_src_adr(pkt),
get_src_port(pkt))
            break
        return i + 1024

    def remove_packet(self, port):
        del self.UdpTable[port - 1024]

    def remove_by_val(self, SrIp, SrPrt):
        for i in range(0, 55500):
            if i in self.UdpTable:
                if self.UdpTable[i] == (SrIp, SrPrt):
                    del self.UdpTable[i]

    def get_src_ip_port(self, port):
        if (port - 1024) in self.UdpTable:
            return self.UdpTable[port - 1024]
        return 0

    def get_and_delete(self, port):
        x = 0
        if (port - 1024) in self.UdpTable:
            x = self.UdpTable[port - 1024]
            self.remove_by_val(x[0], x[1])
        return x

    def printdict(self):
        for key, value in self.UdpTable.items():
            print "-->" + str(key)
            print value[0] + ": " + str(value[1])

```

```

def gui_parse(pkt):
    if get_type(pkt) == 'ICMP':
        returnable = (get_type(pkt), get_src_adr(pkt), '-',
get_dst_adr(pkt), '-')
    else:
        returnable = (get_type(pkt), get_src_adr(pkt),
str(get_src_port(pkt)), get_dst_adr(pkt), str(get_dst_port(pkt)))

    return returnable

def packet_manipulator(pkt):    # Responsible for manipulating given
packet so it can be sent out according to its
                                # arrival interface and protocol

    newp = pkt

    if get_dst_adr(pkt) != get_inf_ip(InterfaceList[1]):
        # From Lan to Wan
        newp = change_dst_mac(newp,
ARPCache.return_value(get_dst_adr(pkt)))
        newp = reset_ip_chk_sum(newp)
        newp = reset_frag(newp)

        if ord(pkt[23]) == 1: # ICMP Protocol
            newp = change_src_adr(newp, get_inf_ip(InterfaceList[1]))
            newp = change_src_mac(newp, get_inf_mac(InterfaceList[1]))
            newp = reset_icmp_check(newp) #Store ICMP details

        elif ord(pkt[23]) == 6: # TCP Protocol
            newp = change_src_port(newp, TcpTable.add_packet(newp))
            newp = change_src_adr(newp, get_inf_ip(InterfaceList[1]))
            newp = change_src_mac(newp, get_inf_mac(InterfaceList[1]))

        elif ord(pkt[23]) == 17: # UDP Protocol
            newp = change_src_port(newp, UdpTable.add_packet(newp))
            newp = change_src_adr(newp, get_inf_ip(InterfaceList[1]))
            newp = change_src_mac(newp, get_inf_mac(InterfaceList[1]))
            newp = reset_udp_check(newp)

        #DNS SPOOF
        #newp = change_dst_mac(newp, get_src_mac(newp))
        #newp = change_src_mac(newp,
ARPCache.return_value('10.8.8.2'))

    else:
        # From Wan to Lan

        newp = change_src_mac(newp, get_inf_mac(InterfaceList[2]))
        newp = reset_ip_chk_sum(newp)
        newp = reset_frag(newp)

        if ord(pkt[23]) == 1: # ICMP Protocol
            newp = reset_icmp_check(newp)
            newp = change_dst_adr(newp, ICMPNat.get_dst_adr(newp))
            newp = change_dst_mac(newp,
ARPCache.return_value(get_dst_adr(newp)))

```

```

        elif ord(pkt[23]) == 6: # TCP Protocol
            if TcpTable.get_src_ip_port(get_dst_port(newp)) != 0:
                newp = change_dst_adr(newp,
                TcpTable.get_src_ip_port(get_dst_port(newp))[0])
                newp = change_dst_mac(newp,
                ARPCache.return_value(get_dst_adr(newp)))
                newp = change_dst_port(newp,
                TcpTable.get_src_ip_port(get_dst_port(newp))[1])
                newp = change_src_adr(newp, '10.8.8.2')

                if 'F' in get_tcp_flags(newp):
                    TcpTable.remove_by_val(get_dst_adr(newp),
                    get_dst_port(newp))

            else:
                newp = ''

        elif ord(pkt[23]) == 17: # UDP Protocol
            newp = change_dst_adr(newp,
            UdpTable.get_src_ip_port(get_dst_port(newp))[0])
            newp = change_dst_mac(newp,
            ARPCache.return_value(get_dst_adr(newp)))
            newp = change_dst_port(newp,
            UdpTable.get_and_delete(get_dst_port(newp))[1])

    return newp

def start_packet_sniffer(interface_name, sniff_method): # Starts
packet sniffer on interface using the IPSniffer class
    packet_sniffer = IPSniff(interface_name, sniff_method)
    packet_sniffer.setDaemon(True)
    packet_sniffer.start()
    logging.debug('Packet sniffer on interface %s started
successfully', interface_name)

def start_packet_sender(interface_name): # Starts packet sender on
interface using the IPSniffer class
    packet_sender = PacketSender(interface_name)
    packet_sender.setDaemon(True)
    packet_sender.start()
    logging.debug('Packet sender on interface %s started successfully',
interface_name)

```

```

if __name__ == "__main__":
    print '(You can always type EXIT to close application)'
    LogSystem = LoggingManager()
    try:
        ArrivedPackets = Queue.Queue(5000) # Arrive packets queue for
        interface eth0
        InterfaceList = os.listdir('/sys/class/net/') # Returns list
        of all networks interfaces of machine
        ARPCache = ARPCache() # Starts ARPCache
        ICMPNat = IcmpNat() # Loads ICMP Nat Table
        TcpTable = TcpNat() # Loads TCP Nat Table
        UdpTable = UdpNat() # Loads UDP Nat Table
        ReturnedPackets = Queue.Queue(5000) # Arrive packets queue for
        interface eth1
    except Exception, message:
        logging.error(message)

    start_packet_sniffer(InterfaceList[2], 'receive_raw_packets') #
    Starts packet sniffers and senders
    start_packet_sniffer(InterfaceList[1], 'receive_raw_packets')
    start_packet_sender(InterfaceList[2])
    start_packet_sender(InterfaceList[1])

    if LogSystem.options['loglevel'] != 'DEBUG' and
    LogSystem.options['loglevel'] != 'ERROR':
        start_nat_monitor()

    while True:
        response = raw_input()
        if response == 'EXIT':
            os._exit(1)

```

NATmonitor.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: NATmonitor                                           #
# Version: 1.0                                               #
# Linux Tested Versions: Centos 7 64 bit                     #
# Python Tested Versions: 2.7.10 32-bit                     #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

import wx
import wx.grid
import threading
import Queue
import time
import datetime

class NATmonitor(wx.Frame, threading.Thread):

    def __init__(self):
        threading.Thread.__init__(self)

    def run(self):
        self.app = wx.PySimpleApp()
        wx.Frame.__init__(self, None, -1, 'NATmonitor', size=(775,
605),
                                style=wx.DEFAULT_FRAME_STYLE &
(~wx.CLOSE_BOX) & (~wx.MAXIMIZE_BOX) ^ wx.RESIZE_BORDER)
        self.panel = wx.Panel(self, pos=(100, 100))
        self.grid = wx.grid.Grid(self.panel, pos=(100, 100))
        self.grid_setup()
        self.start_background('refresh_grid')
        self.app.MainLoop()

    def grid_setup(self):
        self.grid.CreateGrid(14, 6)
        self.grid.SetDefaultCellAlignment(wx.ALIGN_CENTER,
wx.ALIGN_CENTER)
        self.grid.SetDefaultCellFont(wx.Font(10, wx.FONTFAMILY_MODERN,
0, wx.FONTWEIGHT_NORMAL))
        self.grid.SetDefaultCellTextColour('NAVY')
        for j in range(14):
            self.grid.SetCellFont(j, 5, wx.Font(8,
wx.FONTFAMILY_MODERN, 0, wx.FONTWEIGHT_NORMAL))
            sizer = wx.BoxSizer(wx.VERTICAL)
            sizer.Add(self.grid, 1, wx.EXPAND)
            self.panel.SetSizer(sizer)
        for i in range(15):
            self.grid.SetRowSize((i - 1), 40)
```

```

for j in range(6):
    self.grid.SetColSize((j - 1), 120)
self.Show()
self.grid.EnableDragGridSize(False)
self.grid.DisableDragColSize()
self.grid.DisableDragRowSize()
self.grid.SetColLabelValue(0, 'Type')
self.grid.SetColLabelValue(1, 'Source IP')
self.grid.SetColLabelValue(2, 'Source Port')
self.grid.SetColLabelValue(3, 'Destination Ip')
self.grid.SetColLabelValue(4, 'Destination Port')
self.grid.SetColLabelValue(5, 'Time')
self.grid.SetRowLabelValue(0, 'Incoming:')
for i in range(1, 7):
    self.grid.SetRowLabelValue(i, str(i))
self.grid.SetRowLabelValue(7, 'Outgoing:')
for i in range(1, 7):
    self.grid.SetRowLabelValue((i + 7), str(i))

self.grid.SetLabelBackgroundColour('AQUAMARINE')
self.grid.SetLabelTextColour('Purple')
self.grid.SetLabelFont(wx.Font(10, wx.FONTFAMILY_ROMAN, 0,
wx.FONTWEIGHT_MAX))

def refresh_grid(self):
    while True:
        if not entry_buffer.empty():
            x = entry_buffer.get()
            self.add_value(x[0], x[1])

def add_value(self, direction, pkt_tuple):
    try:
        if direction == 'Incoming':
            self.pull_down('in')
            for j in range(5):
                self.grid.SetCellValue(1, j, pkt_tuple[j])
            self.grid.SetCellValue(1, 5,
datetime.datetime.now().strftime("%I:%M:%S%p"))

            if direction == 'Outgoing':
                self.pull_down('out')
                for j in range(5):
                    self.grid.SetCellValue(8, j, pkt_tuple[j])
                self.grid.SetCellValue(8, 5,
datetime.datetime.now().strftime("%I:%M:%S%p"))

            time.sleep(0.7)
    except:
        pass

```

```

def pull_down(self, in_or_out):
    if in_or_out == 'in':
        for i in range(7, 1, -1):
            for j in range(6):
                self.grid.SetCellValue(i, j,
self.grid.GetCellValue((i-1), j))

            for j in range(6):
                self.grid.SetCellValue(1, j, '')

    if in_or_out == 'out':
        for i in range(15, 8, -1):
            for j in range(6):
                if i == 14:
                    self.grid.SetCellValue(i, j, '')
                else:
                    self.grid.SetCellValue(i, j,
self.grid.GetCellValue((i-1), j))

            for j in range(6):
                self.grid.SetCellValue(8, j, '')

        for j in range(6):
            self.grid.SetCellValue(0, j, '')
        for j in range(6):
            self.grid.SetCellValue(7, j, '')

    def start_background(self, func): # helper method to run a
function in another thread
        thread = threading.Thread(target=getattr(self, func))
        thread.setDaemon(True)
        thread.start()

def start_nat_monitor():
    t.setDaemon(True)
    t.start()

def gui_entry(direction, pkt_tuple):
    entry_buffer.put((direction, pkt_tuple))

entry_buffer = Queue.Queue(100)
t = NATmonitor()

```

LoggingSystem.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: LoggingSystem                                       #
# Version: 1.0                                              #
# Linux Tested Versions: Centos 7 64 bit                    #
# Python Tested Versions: 2.7.10 32-bit                     #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

import logging
import os
import sys
import datetime
from optparse import OptionParser

class LoggingManager:

    def __init__(self):

        self.loglevel = self.logging_by_option()

    def option_parsing(self):
        parser = OptionParser()
        parser.add_option("-l", "--log", dest="loglevel",
                          help="Write logging information according to
the selected level")
        parser.add_option("-q", "--quiet",
                          action="store_false", dest="verbose",
default=True,
                          help="don't print status messages to stdout")

        self.options, args = parser.parse_args()
        option_dict = vars(self.options)

        if not option_dict['verbose']:
            f = open(os.devnull, 'w')
            sys.stdout = f

        return option_dict
```

```

def logging_by_option(self):
    self.options = self.option_parsing()
    if self.options['loglevel'] is not None:
        numeric_level = getattr(logging,
self.options['loglevel'].upper(), None)
        if not isinstance(numeric_level, int):
            raise ValueError('Invalid log level: %s' %
self.options['loglevel'])

        if self.options['loglevel'] == 'debug' or
self.options['loglevel'] == 'debug'.upper():
            self.options['loglevel'] = 'DEBUG'
            logging.basicConfig(filename=(self.options['loglevel']
+ '.log'), level=logging.DEBUG,
                                format='%(levelname)s:
%(message)s')
            logging.debug('### ' + str(datetime.datetime.now()) + "
###")
            logging.basicConfig(filename=(self.options['loglevel']
+ '.log'), level=logging.DEBUG,
                                format='%(asctime)s %(message)s: ',
                                datefmt='%m/%d/%Y %I:%M:%S %p')
            log = open("DEBUG.log", "a")
            sys.stdout = log

        elif self.options['loglevel'] == 'error' or
self.options['loglevel'] == 'error'.upper():
            self.options['loglevel'] = 'ERROR'
            logging.basicConfig(filename=(self.options['loglevel']
+ '.log'), level=logging.ERROR,
                                format='%(levelname)s:
%(message)s')
            logging.error('### ' +
datetime.datetime.now().strftime("%A, %d. %B %Y %I:%M%p") + " ###")
            logging.basicConfig(filename=(self.options['loglevel']
+ '.log'), level=logging.ERROR,
                                format='%(asctime)s %(message)s: ',
                                datefmt='%m/%d/%Y %I:%M:%S %p')

        elif self.options['loglevel'] == 'info' or
self.options['loglevel'] == 'info'.upper():
            self.options['loglevel'] = 'INFO'
            logging.basicConfig(filename=(self.options['loglevel']
+ '.log'), level=logging.INFO,
                                format='%(levelname)s:
%(message)s')
            logging.info('### ' +
datetime.datetime.now().strftime("%A, %d. %B %Y %I:%M%p") + " ###")
            logging.basicConfig(filename=(self.options['loglevel']
+ '.log'), level=logging.INFO,
                                format='%(asctime)s %(message)s: ',
                                datefmt='%m/%d/%Y %I:%M:%S %p')
            log = open("INFO.log", "a")
            sys.stdout = log

    return self.options['loglevel']

```

SendRecieve.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: SendRecieve                                         #
# Version: 1.0                                              #
# Linux Tested Versions: Centos 7 64 bit                    #
# Python Tested Versions: 2.7.10 32-bit                     #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

from scapy.all import *
from IpHandler import *
from IcmpHandler import *
from TcpHandler import *
from UdpHandler import *
from scapy.layers.inet import Ether, IP, ICMP, TCP, Raw, UDP
from scapy.layers.dns import *

class PortManager:

    def __init__(self):
        self.PortAr = [True for i in range(55500)]

    def get_open_port(self):
        for i in range(55500):
            if self.PortAr[i]:
                self.PortAr[i] = False
                return i + 1000

    def print_all_open(self):
        print "~~~~~"
        for i in range(55500):
            if self.PortAr[i]:
                print i + 1000

    def send_icmp(pkt):
        sendp(Ether(dst=get_dst_mac(pkt)) / IP(dst=get_dst_adr(pkt),
ttl=get_ttl(pkt), id=int(get_ip_id(pkt), 16)) /
            ICMP(type=icmp_type(pkt), id=get_icmp_id(pkt),
seq=get_icmp_sq_num(pkt)) / Raw(load=pkt[42:]),
            verbose=False)
```

```

def send_tcp(pkt):
    sendp(Ether(dst=get_dst_mac(pkt)) /
          IP(src=get_src_adr(pkt), dst=get_dst_adr(pkt),
             ttl=(get_ttl(pkt)), id=int(get_ip_id(pkt), 16), flags='DF') /
          TCP(sport=get_src_port(pkt), dport=get_dst_port(pkt),
              seq=get_tcp_sq_num(pkt), ack=get_ack_num(pkt),
              flags=get_tcp_flags(pkt), options=get_tcp_options(pkt),
              window=get_window_size(pkt)) /
          Raw(load=get_tcp_raw_load(pkt).decode('unicode_escape')),
            verbose=False)

def send_udp(pkt):
    x = dns_forward(pkt)
    sendp(x, iface='eth1', verbose=False)

def summary_dump(pkt):
    return Ether(pkt).summary()

def get_type(pkt):
    if ord(pkt[23]) == 1:
        return 'ICMP'
    elif ord(pkt[23]) == 6:
        return 'TCP'
    elif ord(pkt[23]) == 17:
        return 'UDP'

```

IpHandler.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: IpHandler                                           #
# Version: 1.0                                              #
# Linux Tested Versions: Centos 7 64 bit                    #
# Python Tested Versions: 2.7.10 32-bit                     #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

import socket
import struct
import fcntl

def get_src_addr(pkt):
    return socket.inet_ntoa(pkt[26:30])

def get_dst_addr(pkt):
    return socket.inet_ntoa(pkt[30:34])

def change_src_addr(pkt, ip_str):
    new_packet = pkt[:26] + socket.inet_aton(ip_str) + pkt[30:]
    return new_packet

def change_dst_addr(pkt, ip_str):
    new_packet = pkt[:30] + socket.inet_aton(ip_str) + pkt[34:]
    return new_packet

def get_src_mac(pkt):
    return str(hex(ord(pkt[6])))[2:] + ":" + str(hex(ord(pkt[7])))[2:]
+ ":" + str(
    hex(ord(pkt[8]))[2:] + ":" + str(hex(ord(pkt[9]))[2:] + ":" +
    str(
    hex(ord(pkt[10]))[2:] + ":" + str(hex(ord(pkt[11]))[2:]

def get_dst_mac(pkt):
    return str(hex(ord(pkt[0])))[2:] + ":" + str(hex(ord(pkt[1]))[2:]
+ ":" + str(
    hex(ord(pkt[2]))[2:] + ":" + str(hex(ord(pkt[3]))[2:] + ":" +
    str(
    hex(ord(pkt[4]))[2:] + ":" + str(hex(ord(pkt[5]))[2:]

def get_inf_mac(interface):
    with open('/sys/class/net/%s/address' % interface) as f:
        mac = f.read()
```

```

    return mac

def get_inf_ip(interface):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    return socket.inet_ntoa(fcntl.ioctl(s.fileno(), 0x8915,
    struct.pack('256s', interface[:15]))[20:24])

def reset_frag(pkt):
    for i in range(20, 22):
        pkt = pkt[:i] + str(chr(0)) + pkt[i+1:]
    return pkt

def get_chk_sum(pkt):
    return '0x' + str(hex(ord(pkt[24])))[2:] +
    str(hex(ord(pkt[25])))[2:]

def reset_ip_chk_sum(pkt):
    for i in range(24, 26):
        pkt = pkt[:i] + str(chr(0)) + pkt[i+1:]
    return pkt

def get_ttl(pkt):
    return int(str(ord(pkt[22])), 10)

def get_ip_id(pkt):
    return (str(hex(int(str(hex(ord(pkt[18])))[2:] +
    str(hex(ord(pkt[19])))[2:], 16))))[2:]

def get_ip_flags(pkt):
    temp = str(bin(int(str(int(pkt[18:20].encode('hex'),
    16))))[2:].zfill(16))[3]
    FlagTuple = ()
    if temp[1] == '1':
        FlagTuple = list(FlagTuple)
        FlagTuple.insert(len(FlagTuple), ('DF'))
        FlagTuple = tuple(FlagTuple)
    elif temp[2] == '1':
        pass
    return ''.join(FlagTuple)

def change_src_mac(pkt, new_mac):
    new_mac += ":"
    for i in range(6, 12):
        temp = new_mac[:new_mac.index(":")]
        pkt = pkt[:i] + chr(int(temp[:2], 16)) + pkt[i + 1:]
        new_mac = new_mac[new_mac.index(":") + 1:]
    return pkt

```

```
def change_dst_mac(pkt, new_mac):  
    new_mac += ":"  
    for i in range(0, 6):  
        temp = new_mac[:new_mac.index(":")]  
        pkt = pkt[:i] + chr(int(temp[:2], 16)) + pkt[i + 1:]  
        new_mac = new_mac[new_mac.index(":") + 1:]  
    return pkt
```

IcmpHandler.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: IcmpHandler                                           #
# Version: 1.0                                                #
# Linux Tested Versions: Centos 7 64 bit                     #
# Python Tested Versions: 2.7.10 32-bit                      #
# Python Environment : PyCharm 2017.1.1                      #
#####
"""

def get_icmp_id(pkt):
    return int((str(hex(int(str(hex(ord(pkt[38]))))[2:] +
str(hex(ord(pkt[39])))[2:], 16))))[2:], 16)

def get_icmp_sq_num(pkt):
    return int(pkt[40:42].encode('hex'), 16)

def icmp_type(pkt):
    type_file = open('ICMPTypes.txt', 'r')
    for line in type_file.readlines():
        if line.find(str(hex(ord(pkt[34])))[2:]) != -1:
            return line[line.find(':') + 1:line.index("\n")]

def reset_icmp_check(pkt):
    for i in range(36, 38):
        pkt = pkt[:i] + str(chr(0)) + pkt[i+1:]
    return pkt
```

TcpHandler.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: TcpHandler                                           #
# Version: 1.0                                               #
# Linux Tested Versions: Centos 7 64 bit                     #
# Python Tested Versions: 2.7.10 32-bit                      #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

import ast
from StringIO import StringIO
import sys
from scapy.layers.inet import Ether

def show_dump(pkt):
    capture = StringIO()
    save_stdout = sys.stdout
    sys.stdout = capture
    Ether(pkt).show()
    sys.stdout = save_stdout

    # capture.getvalue() is a string with the output of 'pack.show()'
    return capture.getvalue()

def get_tcp_options(pkt):
    if 'Raw' not in show_dump(pkt):
        options = show_dump(pkt)[show_dump(pkt).find("options",
show_dump(pkt).find("sport")):][12:]
    else:
        options = show_dump(pkt)[show_dump(pkt).find("options",
show_dump(pkt).find("sport")):show_dump(pkt).find('Raw') - 6][12:]
    return ast.literal_eval(options)

def get_tcp_raw_load(pkt):
    str_raw = show_dump(pkt)
    if 'load' in str_raw:
        str_raw = show_dump(pkt)[show_dump(pkt).find("load") +
13:show_dump(pkt).rfind("")]
    else:
        str_raw = ''
    return str_raw

def get_tcp_sq_num(pkt):
    return int(pkt[38:42].encode('hex'), 16)

def get_ack_num(pkt):
```

```

    return int(pkt[42:46].encode('hex'), 16)

def get_data_off(pkt):
    return str(bin(int(str(int(pkt[46:48].encode('hex'),
16))))[2:].zfill(4))[0:2]

def get_tcp_flags(pkt):
    temp = str(bin(int(str(int(pkt[46:48].encode('hex'),
16))))[2:].zfill(16))[7:]
    flag_tuple = ()
    if temp[0] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'N')
        flag_tuple = tuple(flag_tuple)
    if temp[1] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'C')
        flag_tuple = tuple(flag_tuple)
    if temp[2] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'E')
        flag_tuple = tuple(flag_tuple)
    if temp[3] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'U')
        flag_tuple = tuple(flag_tuple)
    if temp[4] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'A')
        flag_tuple = tuple(flag_tuple)
    if temp[5] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'P')
        flag_tuple = tuple(flag_tuple)
    if temp[6] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'R')
        flag_tuple = tuple(flag_tuple)
    if temp[7] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'S')
        flag_tuple = tuple(flag_tuple)
    if temp[8] == '1':
        flag_tuple = list(flag_tuple)
        flag_tuple.insert(len(flag_tuple), 'F')
        flag_tuple = tuple(flag_tuple)
    return ''.join(flag_tuple)

def reset_tcp_check(pkt):
    for i in range(40, 42):
        pkt = pkt[:i] + str(chr(0)) + pkt[i+1:]
    return pkt

```

```

def get_window_size(pkt):
    return int(pkt[48:50].encode('hex'), 16)

def get_src_port(pkt):
    return int(pkt[34:36].encode('hex'), 16)

def get_dst_port(pkt):
    return int(pkt[36:38].encode('hex'), 16)

def change_src_port(pkt, new_packet):
    new_packet = format(new_packet, 'x').zfill(4)
    for i in range(34, 36):
        temp = new_packet[:2]
        pkt = pkt[:i] + chr(int(temp[:2], 16)) + pkt[i + 1:]
        new_packet = new_packet[2:]
    return pkt

def change_dst_port(pkt, new_packet):
    new_packet = format(new_packet, 'x').zfill(4)
    for i in range(36, 38):
        temp = new_packet[:2]
        pkt = pkt[:i] + chr(int(temp[:2], 16)) + pkt[i + 1:]
        new_packet = new_packet[2:]
    return pkt

```

UdpHandler.py

```
"""
#####
# Created By: Kobie Hazon                                     #
# Date: 04/05/2017                                           #
# Name: UdpHandler                                           #
# Version: 1.0                                               #
# Linux Tested Versions: Centos 7 64 bit                     #
# Python Tested Versions: 2.7.10 32-bit                      #
# Python Environment : PyCharm 2017.1.1                     #
#####
"""

import dpkt
from scapy.layers.inet import Ether, IP, ICMP, TCP, Raw, UDP
from scapy.layers.dns import *

def get_src_port(pkt):
    return int(pkt[34:36].encode('hex'), 16)

def get_dst_port(pkt):
    return int(pkt[36:38].encode('hex'), 16)

def change_src_port(pkt, newp):
    newp = format(newp, 'x').zfill(4)
    for i in range(34, 36):
        temp = newp[:2]
        pkt = pkt[:i] + chr(int(temp[:2], 16)) + pkt[i + 1:]
        newp = newp[2:]
    return pkt

def change_dst_port(pkt, newp):
    newp = format(newp, 'x').zfill(4)
    for i in range(36, 38):
        temp = newp[:2]
        pkt = pkt[:i] + chr(int(temp[:2], 16)) + pkt[i + 1:]
        newp = newp[2:]
    return pkt

def reset_udp_check(pkt):
    for i in range(40, 42):
        pkt = pkt[:i] + str(chr(0)) + pkt[i+1:]
    return pkt
```

```

def dns_forward(pkt):
    eth = dpkt.ethernet.Ethernet(pkt)
    ip = eth.data
    udp = ip.data
    dns = dpkt.dns.DNS(udp.data)
    x = Ether(str(eth))
    return x

def dns_spoof(pkt):
    eth = dpkt.ethernet.Ethernet(pkt)
    ip = eth.data
    udp = ip.data
    dns = dpkt.dns.DNS(udp.data)
    # transform DNS query into response
    dns.op = dpkt.dns.DNS_RA
    dns.rcode = dpkt.dns.DNS_RCODE_NOERR
    dns.qr = dpkt.dns.DNS_R

    # construct our fake answer RR
    arr = dpkt.dns.DNS.RR()
    arr.cls = dpkt.dns.DNS_IN
    arr.type = dpkt.dns.DNS_A
    arr.name = 'wan.kobie.com'
    arr.ip = socket.inet_aton('10.8.8.2')

    dns.an.append(arr)

    # fix up IP and UDP layers
    udp.sport, udp.dport = udp.dport, udp.sport
    ip.src, ip.dst = ip.dst, ip.src
    udp.data = dns
    udp.ulen = len(udp)
    ip.len = len(ip)
    x = Ether(str(eth))
    return x

```

NATscript.sh

```
#!/bin/bash

trap "" HUP

iptables -A FORWARD -j DROP
iptables -A OUTPUT -p tcp -s 10.8.8.1 -d 10.8.8.2 --dport 80 -
j DROP
/sbin/service iptables save > /dev/null
systemctl restart iptables

trap - HUP
echo -e "Enter NAT program parameters:"
read parameters
python Main.py $parameters
```

ביבליוגרפיה

- "רשתות מחשבים", עומר רוזנבוים ושלומי הוד, מטה הסייבר העולמי – cyber.org.il. דרך ספר זה למדתי בתחילת שנה זו ומשם קיבלתי את הבסיס לידע שלי ברשתות בו נעזרתי בעת פיתוח פרויקט זה.
- "Python", נועה זילברמן, מטה הסייבר העולמי – cyber.org.il. מספר זה למדתי את בסיסי ידעי בפייתון בתחילת השנה שממנו התפחתי והתקדמתי על מנת לכתוב פרויקט זה.
- המדריך הרשמי לפייתון 2.7 - <https://docs.python.org/2/howto/>. מכיל מידע נרחב על השימוש בשפה פייתון ובמודולים רבים.
- <http://www.tcpipguide.com/index.htm>, ספר אינטרנטי נרחב על הפרוטוקולים תחת חבילת הפרוטוקולים של TCP/IP המכיל דוגמאות, תרשימים וטבלאות השוואה רבות. ממנו למדתי את המבנה והשימוש בפתיחי מנות המידע.
- <https://wiki.wxpython.org/How%20to%20Learn%20wxPython>, מדריך לשימוש במודול WxPython לצורך בניית הממשק הגרפי (GUI) של הפרויקט שלי.

